

Migración Reporte Gasolina VB6 a C# WinForms

Información General

Proyecto: Reporte Gasolina

Origen: Visual Basic 6.0

Destino: C# WinForms

Fecha de Inicio: 05/08/2026

Estado General: En análisis y planificación

Avance Estimado: 15%

Objetivo

Migrar el sistema de Reporte de Gasolina desarrollado en VB6 a C# WinForms, conservando la lógica de negocio, consultas SQL, procedimientos almacenados y generación de reportes Excel.

Alcance Identificado

Módulos VB6 Analizados

- frmRepGas.frm
- Conexion.frm
- RepGas.bas
- clsMAdoNuevo.cls
- RepGas.vbp

Funcionalidades Detectadas

Consulta de precios

Consulta de precios de gasolina por ciudad.

Carga Excel

Importación masiva de precios desde archivo Excel.

Proceso de cálculo

Generación de información de asignación de gasolina.

Exportación Excel

Generación de reportes detallados:

- General
 - Altas
 - Faltas
 - Incapacidades
-

Decisiones Técnicas

DEC-001

Fecha: 05/08/2026

Decisión: Eliminar frmConexion.

Motivo:

La conexión será administrada mediante configuración centralizada.

Impacto:

- No habrá pantalla de conexión.
 - No habrá captura manual de DSN.
 - No habrá captura manual de usuario y contraseña.
-

DEC-002

Fecha: 05/08/2026

Decisión: Sustituir parámetros de conexión por App.config.

Impacto:

La aplicación obtendrá:

- ConnectionString
- Usuario
- Compañía
- Operación

Desde App.config.

DEC-003

Fecha: 05/08/2026

Decisión: Implementar arquitectura por capas.

Estructura objetivo:

Infrastructure/

- DatabaseManager.cs

Models/

- PrecioGasolinaModel.cs
- ReporteGasolinaModel.cs

Services/

- SeguridadService.cs
- GasolinaService.cs
- ExcelService.cs

Utilities/

- Globals.cs
- AppSettingsHelper.cs

Forms/

- FrmReporteGasolina.cs
-

Inventario de Objetos SQL

Consultas

Tabla

Ls_HistPrecioGasolinaTbl

Uso:

- Consulta de precios por ciudad.

Tabla

Ls_RepPrecioGasolinaTbl

Uso:

- Reporte principal de asignación de gasolina.
-

Procedimientos Almacenados

spa_Ls_HistPrecioGasolinaTbl

Alta de precios.

spv_Ls_HistPrecioGasolinaTbl

Validación de precios.

spd_Ls_HistPrecioGasolinaTbl

Depuración previa a carga.

spp_Ls_RepPrecioGasolinaTbl

Proceso de generación del reporte.

Equivalencias VB6 → C#

VB6	C#
frmConexion	App.config + SeguridadService
clsMAdoNuevo	DatabaseManager
Recordset	DataTable
ListView	DataGridView
Globals	Globals.cs
fEjecuta	DatabaseManager
CommonDialog	OpenFileDialog
CreateObject("Excel.Application")	ClosedXML o EPPlus

Fases del Proyecto

Fase 1 - Infraestructura

Pendiente

Entregables

- DatabaseManager.cs
 - Globals.cs
 - AppSettingsHelper.cs
-

Fase 2 - Seguridad

Pendiente

Entregables

- SeguridadService.cs
 - Validación de compañía
 - Validación de operación
 - Obtención de nivel de seguridad
-

Fase 3 - Consulta de Precios

Pendiente

Entregables

- Migración cmdConsulta_Click
 - DataGridView
 - Consulta Ls_HistPrecioGasolinaTbl
-

Fase 4 - Carga Excel

Pendiente

Entregables

- Lectura Excel
 - Validación de registros
 - Alta masiva
-

Fase 5 - Proceso de Reporte

Pendiente

Entregables

- Migración procesaReporte()
 - Integración spp_Ls_RepPrecioGasolinaTbl
-

Fase 6 - Exportación Excel

Pendiente

Entregables

- Reporte General
 - Altas
 - Faltas
 - Incapacidades
 - Formatos y logotipo
-

Pendientes Inmediatos

1. Revisar código actual de FrmReporteGasolina.cs.
 2. Revisar Program.cs.
 3. Revisar DatabaseManager.cs (si ya existe).
 4. Implementar Fase 1.
 5. Implementar Fase 2.
-

Análisis del Back-End SQL

Plataforma

- SQL Server 2008 R2
- Base de datos: adam
- Servidor: 192.168.1.24:1433
- Operación de seguridad: FPLS001

Tablas Principales Identificadas

Ls_HistPrecioGasolinaTbl

Histórico de precios de gasolina por:

- Compañía
- Año
- Mes
- Ciudad

Ls_RepPrecioGasolinaTbl

Tabla temporal de trabajo utilizada para generar el reporte final de gasolina.

Contiene:

- Región
- Departamento
- Zona
- Ciudad
- Trabajador
- NSS
- Litros
- Precio por litro
- Importe mensual
- Faltas
- Incapacidades
- Neto a pagar
- Tarjeta de depósito

Ls_FaltasGasolinaTbl

Tabla temporal para consolidar faltas que afectan el pago de gasolina.

Ls_FaltasIncapacidadTbl

Tabla temporal para consolidar incapacidades utilizadas en el cálculo de gasolina.

Funciones SQL Identificadas

- fn_desc_agrup_dato
 - fn_obten_nss
 - fn_obten_FinMes
 - fn_obten_faltasGasolina
 - fn_obten_faltasIncapacidad
 - fn_obten_valor_vartrab
 - fn_flagfaltasIncapacidad
-

Procedimientos Almacenados Críticos

Precios de gasolina

- spa_Ls_HistPrecioGasolinaTbl
- spv_Ls_HistPrecioGasolinaTbl
- spd_Ls_HistPrecioGasolinaTbl

Faltas

- spp_Ls_FaltasGasolinaTbl
- spd_Ls_FaltasGasolinaTbl

Incapacidades

- spp_Ls_FaltasIncapacidadTbl
- spd_Ls_FaltasIncapacidadTbl

Reporte principal

- spp_Ls_RepPrecioGasolinaTbl
 - spd_Ls_RepPrecioGasolinaTbl
-

Reglas de Negocio Detectadas

RN-001

Los precios de gasolina son únicos por:

- Compañía
- Año
- Mes
- Ciudad

RN-002

El cálculo base es:

Importe Gasolina = Precio Litro × Litros Asignados

RN-003

Las faltas descuentan proporcionalmente:

Importe Falta = (Importe Gasolina / 30) × Días Falta

RN-004

Las incapacidades pueden reintegrar importe dependiendo del historial de meses anteriores.

RN-005

Existen trabajadores marcados como EXENTO mediante variable de trabajador.

RN-006

La tarjeta de depósito proviene de una variable de trabajador configurable.

Bitácora

05/08/2026

- Analizado el proyecto VB6 completo.
- Identificadas funcionalidades principales.
- Definida arquitectura objetivo.
- Eliminado frmConexion de la estrategia de migración.
- Se adopta App.config como mecanismo de configuración.
- Creado documento maestro del proyecto.
- Analizado el back-end SQL completo.
- Identificadas tablas, funciones, triggers y procedimientos almacenados.
- Confirmado que la lógica principal de negocio reside en spp_Ls_RepPrecioGasolinaTbl.
- Confirmado que la mayor parte del cálculo permanecerá en SQL Server y C# actuará como capa de presentación y orquestación.

Avances adicionales registrados

Infraestructura

- ✓ App.config configurado.
- ✓ AppSettings.cs implementado.
- ✓ SqlHelper.cs implementado con:
 - Execute()
 - ExecuteScalar()
 - ExecuteNonQuery()
- ✓ Conexión exitosa a SQL Server.

✓ Lectura correcta de parámetros:

- Compañía
- Operación
- Usuario
- ConnectionString

Formulario Principal

✓ FrmReporteGasolina creado.

✓ Diseño principal iniciado.

✓ Encabezado implementado:

- Fecha de proceso
- Operación
- Usuario

✓ Botonera principal implementada:

- Consultar
- Importar Excel
- Excel
- Procesar
- Salir

✓ Iconos cargados en la botonera.

✓ ToolTips configurados.

✓ Botón Salir implementado y funcional.

Período de Proceso

✓ Implementada rutina CargarPeriodoActual().

✓ Obtención automática de:

- Año máximo
- Mes máximo

Desde Ls_HistPrecioGasolinaTbl.

✓ Carga automática del período al abrir la aplicación.

Consulta de Precios

✓ Implementado botón Consultar.

✓ Consulta parametrizada por:

- Compañía
- Año
- Mes

✓ Consulta contra:

SELECT ciudad, precio FROM Ls_HistPrecioGasolinaTbl

✓ Carga correcta de resultados en DataGridView.

✓ Consulta automática al abrir la pantalla.

Estado Actual del Proyecto

Infraestructura 100%

Pantalla Principal 90%

Consulta de Precios 100%

Carga Excel Pendiente

Proceso Gasolina Pendiente

Exportación Excel Pendiente

Próximo Desarrollo

FASE 3 - Carga Excel

Implementar:

- btnImportarExcel_Click
- Lectura mediante ClosedXML
- Validación de período
- Validación de ciudades
- Alta de precios
- Actualización automática del grid

Regla de Continuidad

Al iniciar una nueva sesión, cargar este documento como referencia principal y continuar desde la última fase registrada, actualizando decisiones, avance y bitácora.

Consideración adicional: Generación de reportes finales en Excel

La migración no debe limitarse únicamente a la lógica de cálculo y consulta. También debe contemplar la generación de los reportes finales en Excel, incluyendo:

- Replicación del formato actual de salida.
- Encabezados, agrupaciones, totales y subtotales.
- Formatos numéricos, monetarios y de fecha.
- Manejo de múltiples hojas cuando aplique.
- Compatibilidad con archivos XLSX.
- Exportación automática desde la aplicación C#.
- Validación de que el resultado generado coincida con los reportes actuales de VB6.
- Evaluación de librerías para Excel (OpenXML, EPPlus, ClosedXML u otra alternativa aprobada).

Este componente debe considerarse parte del alcance de la migración, ya que representa el entregable final consumido por los usuarios.